

Bank Marketing Classification Project

Atharva Rane

Introduction

In this project, we analyze the **Bank Marketing** dataset (`bank-additional.csv`) from the UCI Machine Learning Repository.

The goal is to predict whether a client subscribes to a **term deposit** (y : **yes/no**) based on client attributes, previous campaign outcomes, and macroeconomic indicators.

We:

- Explore and visualize the data.
- Fit multiple supervised learning models:
 - Logistic regression
 - Penalized logistic regression (LASSO)
 - Classification tree
 - Random forest
- Compare predictive performance using a train–test split.
- Interpret predictor contributions via coefficients and variable importance.
- Assess model stability using bootstrap resampling of the AUC.

Data Loading and Preprocessing

```
library(corrplot)
library(tidyverse)
library(pROC)
library(rpart)
library(rpart.plot)
```

```

library(randomForest)
library(glmnet)
library(boot)

# Read data
bank <- read.csv("bank-additional.csv", sep = ";")

# Response as factor with fixed levels
bank$y <- factor(bank$y, levels = c("no", "yes"))

# Drop duration and default (duration is post-call, default is ambiguous/sparse)
bank <- bank %>%
  select(-duration, -default)

# Make categorical variables factors
bank <- bank %>%
  mutate(
    job      = factor(job),
    marital  = factor(marital),
    education = factor(education),
    housing  = factor(housing),
    loan     = factor(loan),
    contact  = factor(contact),
    month    = factor(month),
    day_of_week = factor(day_of_week),
    poutcome = factor(poutcome)
  )

# Quick structure and outcome balance
str(bank)

```

```

'data.frame':  4119 obs. of  19 variables:
 $ age      : int  30 39 25 38 47 32 32 41 31 35 ...
 $ job      : Factor w/ 12 levels "admin.", "blue-collar",...: 2 8 8 8 1 8 1 3 8 2 ...
 $ marital  : Factor w/ 4 levels "divorced", "married",...: 2 3 2 2 2 3 3 2 1 2 ...
 $ education : Factor w/ 8 levels "basic.4y", "basic.6y",...: 3 4 4 3 7 7 7 7 6 3 ...
 $ housing  : Factor w/ 3 levels "no", "unknown",...: 3 1 3 2 3 1 3 3 1 1 ...
 $ loan     : Factor w/ 3 levels "no", "unknown",...: 1 1 1 2 1 1 1 1 1 1 ...
 $ contact  : Factor w/ 2 levels "cellular", "telephone": 1 2 2 2 1 1 1 1 1 2 ...
 $ month    : Factor w/ 10 levels "apr", "aug", "dec",...: 7 7 5 5 8 10 10 8 8 7 ...
 $ day_of_week : Factor w/ 5 levels "fri", "mon", "thu",...: 1 1 5 1 2 3 2 2 4 3 ...
 $ campaign : int   2 4 1 3 1 3 4 2 1 1 ...

```

```

$ pdays          : int   999 999 999 999 999 999 999 999 999 999 ...
$ previous        : int    0 0 0 0 0 2 0 0 1 0 ...
$ poutcome        : Factor w/ 3 levels "failure","nonexistent",...: 2 2 2 2 2 1 2 2 1 2 ...
$ emp.var.rate    : num   -1.8 1.1 1.4 1.4 -0.1 -1.1 -1.1 -0.1 -0.1 1.1 ...
$ cons.price.idx  : num    92.9 94 94.5 94.5 93.2 ...
$ cons.conf.idx   : num   -46.2 -36.4 -41.8 -41.8 -42 -37.5 -37.5 -42 -42 -
36.4 ...
$ euribor3m       : num    1.31 4.86 4.96 4.96 4.19 ...
$ nr.employed     : num   5099 5191 5228 5228 5196 ...
$ y               : Factor w/ 2 levels "no","yes": 1 1 1 1 1 1 1 1 1 1 ...

```

```
table(bank$y)
```

```

no  yes
3668 451

```

```
prop.table(table(bank$y))
```

```

      no      yes
0.8905074 0.1094926

```

Exploratory Data Analysis

Class balance

```
table(bank$y)
```

```

no  yes
3668 451

```

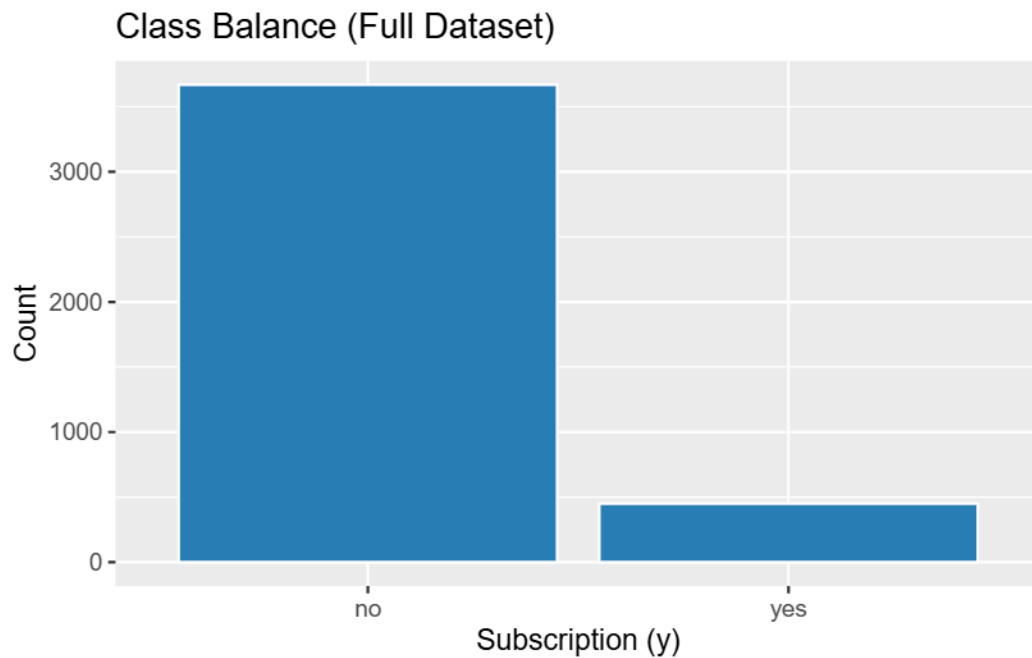
```
prop.table(table(bank$y))
```

```

      no      yes
0.8905074 0.1094926

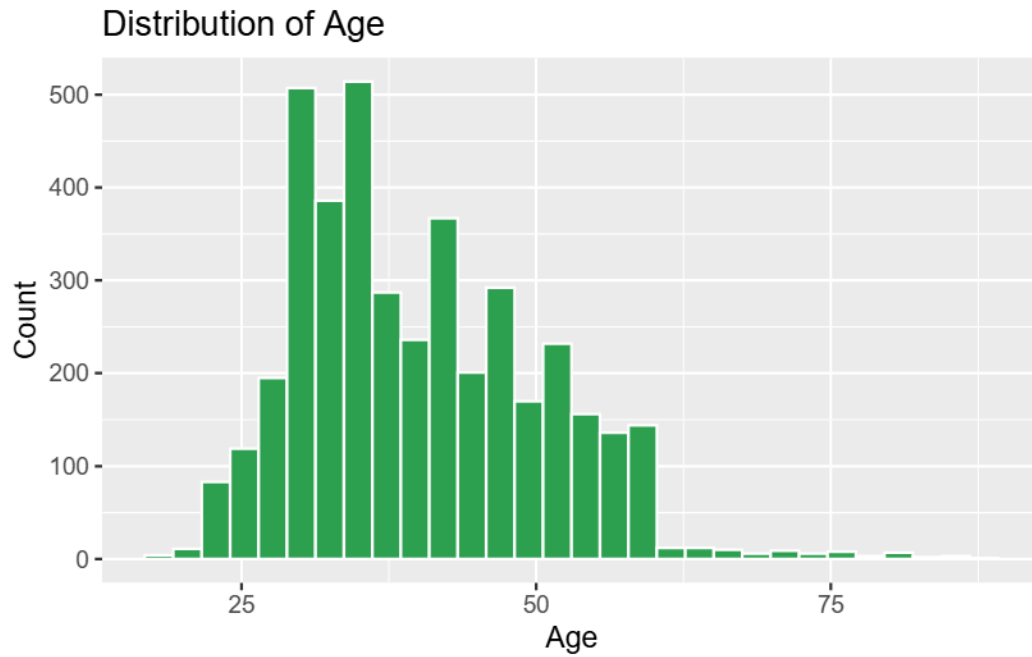
```

```
ggplot(bank, aes(x = y)) +
  geom_bar(fill = "#2C7FB8", color = "white") +
  ggtitle("Class Balance (Full Dataset)") +
  xlab("Subscription (y)") +
  ylab("Count")
```

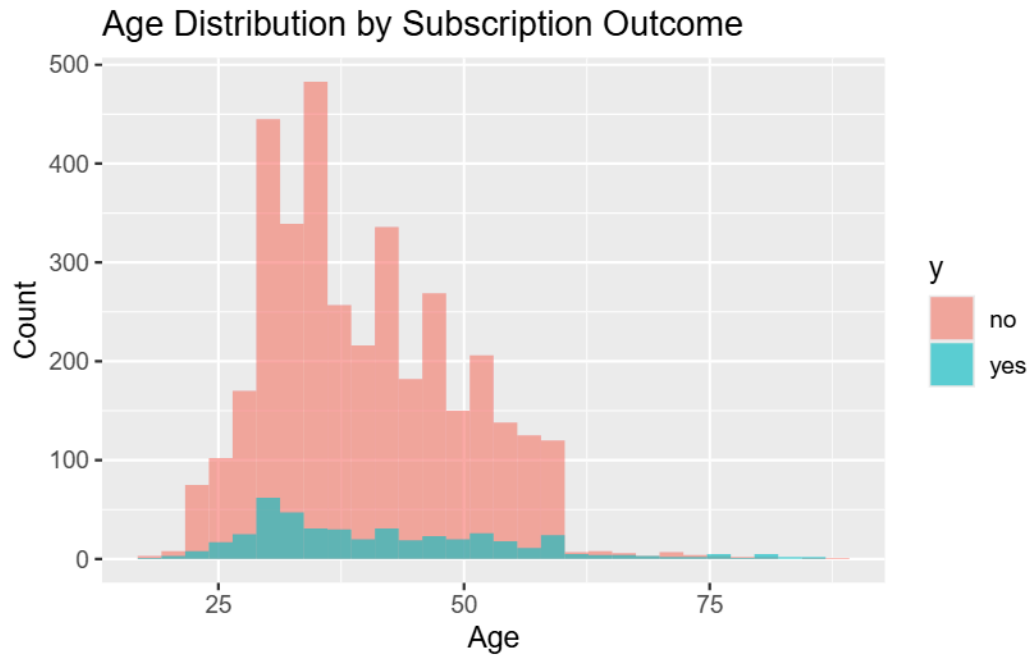


Histogram of Age

```
ggplot(bank, aes(x = age)) +
  geom_histogram(bins = 30, fill = "#31A354", color = "white") +
  ggtitle("Distribution of Age") +
  xlab("Age") +
  ylab("Count")
```

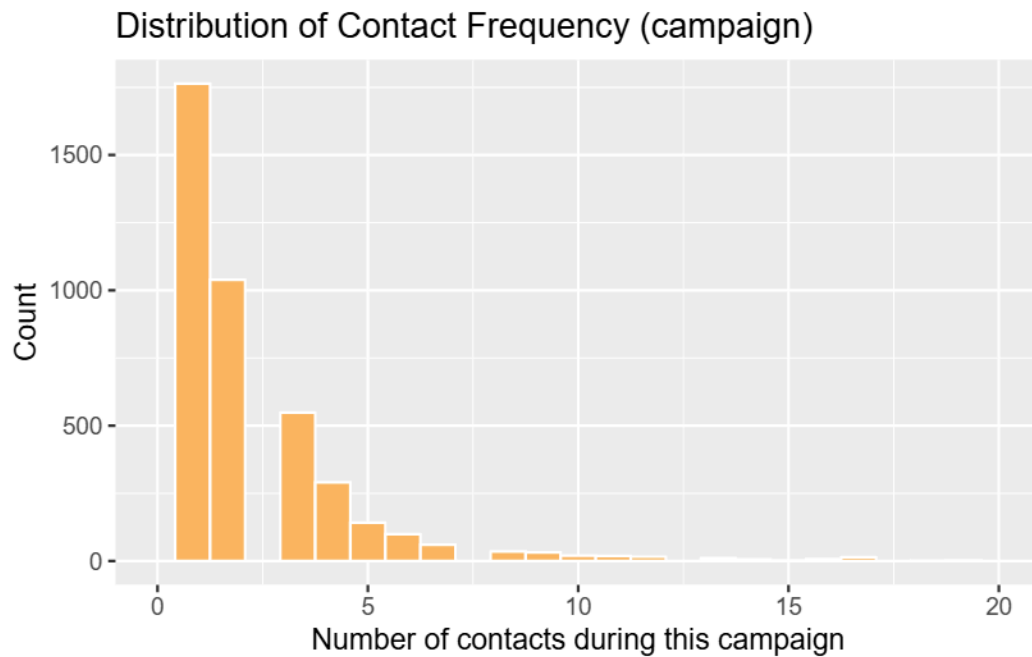


```
ggplot(bank, aes(x = age, fill = y)) +  
  geom_histogram(position = "identity", alpha = 0.6, bins = 30) +  
  ggtitle("Age Distribution by Subscription Outcome") +  
  xlab("Age") +  
  ylab("Count")
```



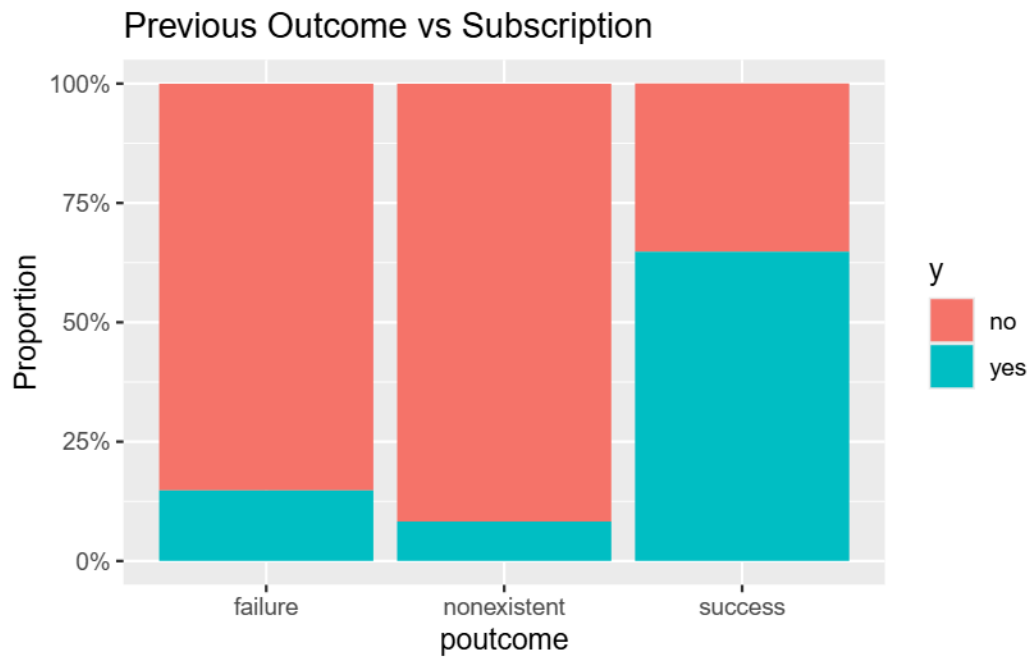
Histogram of Contact Campaign

```
ggplot(bank, aes(x = campaign)) +  
  geom_histogram(bins = 25, fill = "#FDB863", color = "white") +  
  xlim(0, 20) +  
  ggtitle("Distribution of Contact Frequency (campaign)") +  
  xlab("Number of contacts during this campaign") +  
  ylab("Count")
```



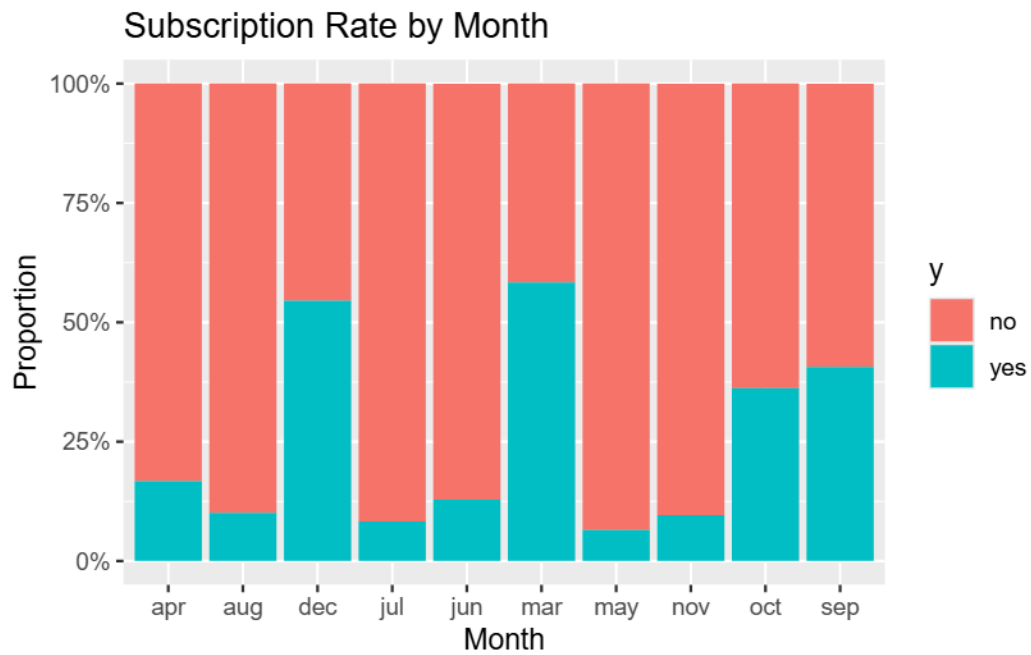
Previous Outcome vs subscriptions

```
ggplot(bank, aes(x = poutcome, fill = y)) +  
  geom_bar(position = "fill") +  
  ggtitle("Previous Outcome vs Subscription") +  
  ylab("Proportion") +  
  xlab("poutcome") +  
  scale_y_continuous(labels = scales::percent)
```



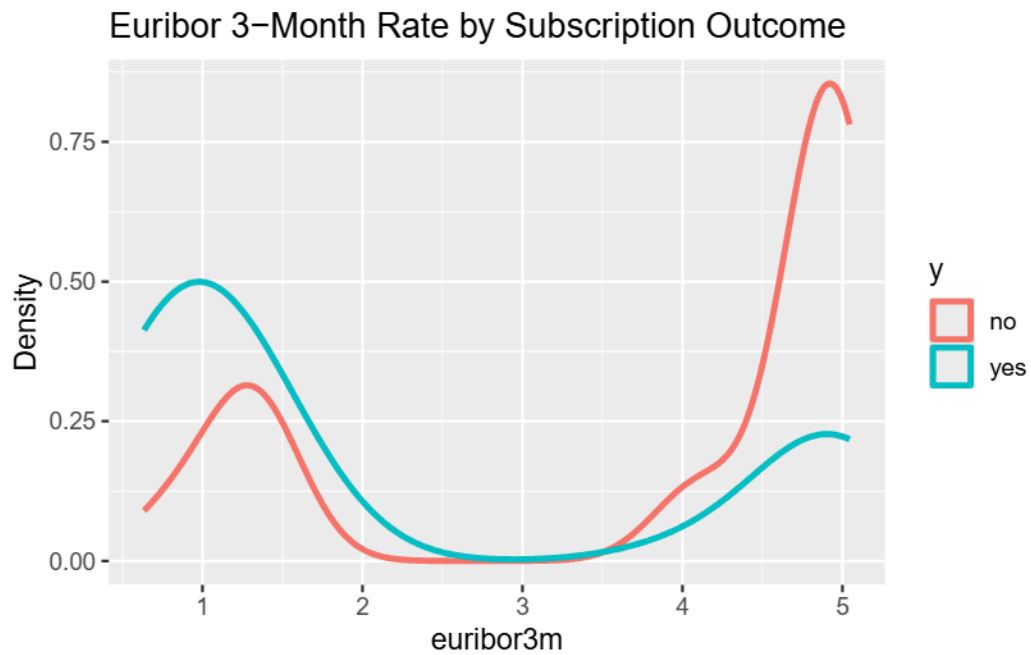
Month vs subscription rate

```
ggplot(bank, aes(x = month, fill = y)) +  
  geom_bar(position = "fill") +  
  ggtitle("Subscription Rate by Month") +  
  ylab("Proportion") +  
  xlab("Month") +  
  scale_y_continuous(labels = scales::percent)
```

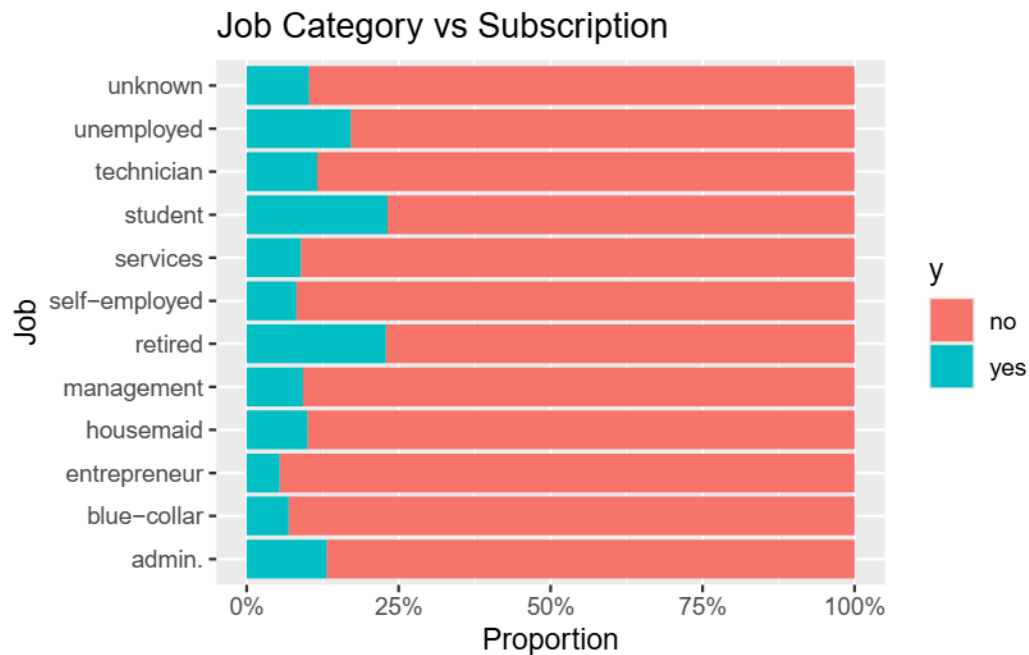
Euribor rate vs subscription (density plot)

```
ggplot(bank, aes(x = euribor3m, color = y)) +  
  geom_density(linewidth = 1.1) +  
  ggtitle("Euribor 3-Month Rate by Subscription Outcome") +  
  xlab("euribor3m") +  
  ylab("Density")
```



Job vs subscription (proportions)

```
ggplot(bank, aes(x = job, fill = y)) +  
  geom_bar(position = "fill") +  
  coord_flip() +  
  ggtitle("Job Category vs Subscription") +  
  ylab("Proportion") +  
  xlab("Job") +  
  scale_y_continuous(labels = scales::percent)
```



```
# Train-test split (70/30)
set.seed(123)
n <- nrow(bank)
train_idx <- sample(1:n, 0.7 * n)
train <- bank[train_idx, ]
test <- bank[-train_idx, ]

prop.table(table(train$y))
```

```
      no      yes
0.8858828 0.1141172
```

```
prop.table(table(test$y))
```

```
      no      yes
0.9012945 0.0987055
```

Logistic Regression baseline model

```
logit_fit <- glm(y ~ ., data = train, family = binomial)
summary(logit_fit)
```

Call:

```
glm(formula = y ~ ., family = binomial, data = train)
```

Coefficients: (1 not defined because of singularities)

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	-2.586e+02	1.235e+02	-2.094	0.036261	*
age	1.464e-02	8.237e-03	1.778	0.075476	.
jobblue-collar	-2.223e-01	2.630e-01	-0.845	0.397943	
jobentrepreneur	-9.236e-01	5.438e-01	-1.699	0.089409	.
jobhousemaid	1.102e-01	4.650e-01	0.237	0.812728	
jobmanagement	-4.919e-01	3.130e-01	-1.572	0.115999	
jobretired	-2.175e-02	3.555e-01	-0.061	0.951208	
jobself-employed	-5.687e-01	3.998e-01	-1.422	0.154919	
jobservices	1.296e-01	2.667e-01	0.486	0.626992	
jobstudent	-5.723e-02	4.295e-01	-0.133	0.893992	
jobtechnician	-1.198e-01	2.259e-01	-0.530	0.595784	
jobunemployed	8.749e-02	3.961e-01	0.221	0.825194	
jobunknown	-1.076e+00	9.035e-01	-1.191	0.233554	
maritalmarried	9.376e-02	2.336e-01	0.401	0.688150	
maritalsingle	4.110e-01	2.635e-01	1.560	0.118805	
maritalunknown	6.175e-01	1.230e+00	0.502	0.615768	
educationbasic.6y	1.433e-03	4.168e-01	0.003	0.997258	
educationbasic.9y	2.365e-01	3.051e-01	0.775	0.438176	
educationhigh.school	-6.324e-02	3.043e-01	-0.208	0.835366	
educationilliterate	-1.120e+01	3.247e+02	-0.034	0.972487	
educationprofessional.course	2.780e-01	3.231e-01	0.861	0.389508	
educationuniversity.degree	1.118e-01	3.051e-01	0.366	0.714042	
educationunknown	1.216e-01	4.097e-01	0.297	0.766675	
housingunknown	-1.268e-01	4.420e-01	-0.287	0.774278	
housingyes	-8.672e-02	1.380e-01	-0.628	0.529717	
loanunknown	NA	NA	NA	NA	
loanyes	4.028e-02	1.831e-01	0.220	0.825934	
contacttelephone	-9.787e-01	2.865e-01	-3.416	0.000634	***
monthaug	-1.309e-01	4.246e-01	-0.308	0.757851	
monthdec	7.366e-01	7.088e-01	1.039	0.298652	

monthjul	-3.165e-01	3.514e-01	-0.901	0.367742	
monthjun	-4.039e-01	4.357e-01	-0.927	0.353968	
monthmar	1.900e+00	5.616e-01	3.384	0.000716	***
monthmay	-3.845e-01	2.969e-01	-1.295	0.195235	
monthnov	-8.249e-01	4.166e-01	-1.980	0.047705	*
monthoct	3.465e-01	5.227e-01	0.663	0.507321	
monthsep	2.251e-01	5.929e-01	0.380	0.704219	
day_of_weekmon	3.364e-02	2.156e-01	0.156	0.876017	
day_of_weekthu	1.122e-01	2.164e-01	0.518	0.604211	
day_of_weektue	-7.084e-02	2.263e-01	-0.313	0.754298	
day_of_weekwed	2.908e-01	2.194e-01	1.325	0.185024	
campaign	-7.542e-02	3.780e-02	-1.995	0.045991	*
pdays	2.817e-04	7.488e-04	0.376	0.706749	
previous	1.305e-01	2.042e-01	0.639	0.522793	
poutcomenonexistent	5.800e-01	3.318e-01	1.748	0.080450	.
poutcomesuccess	2.177e+00	7.483e-01	2.909	0.003630	**
emp.var.rate	-1.215e+00	4.642e-01	-2.616	0.008886	**
cons.price.idx	2.161e+00	8.188e-01	2.640	0.008294	**
cons.conf.idx	6.153e-02	2.792e-02	2.204	0.027524	*
euribor3m	-1.132e-01	4.180e-01	-0.271	0.786531	
nr.employed	1.087e-02	9.950e-03	1.092	0.274790	

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 2047.1 on 2882 degrees of freedom
 Residual deviance: 1607.7 on 2833 degrees of freedom
 AIC: 1707.7

Number of Fisher Scoring iterations: 11

```
logit_prob <- predict(logit_fit, test, type = "response")
logit_pred <- ifelse(logit_prob > 0.5, "yes", "no") |> factor(levels=c("no", "yes"))

table(logit_pred, test$y)
```

logit_pred	no	yes
no	1088	91
yes	26	31

```
logit_auc <- auc(roc(test$y, logit_prob))
logit_auc
```

Area under the curve: 0.8072

Penalized Logistic Regression (LASSO)

```
x_train <- model.matrix(y ~ . - 1, train)
y_train <- train$y

set.seed(123)
cv_lasso <- cv.glmnet(x_train, y_train, family="binomial", alpha=1)

best_lambda <- cv_lasso$lambda.min
lasso_fit <- glmnet(x_train, y_train, family="binomial", alpha=1, lambda=best_lambda)

x_test <- model.matrix(y ~ . - 1, test)
lasso_prob <- predict(lasso_fit, newx = x_test, type="response")[,1]
lasso_pred <- ifelse(lasso_prob > 0.5, "yes", "no") |> factor(levels=c("no", "yes"))

table(lasso_pred, test$y)
```

lasso_pred	no	yes
no	1096	96
yes	18	26

```
lasso_auc <- auc(roc(test$y, lasso_prob))
lasso_auc
```

Area under the curve: 0.8185

Ridge Regression

```

# Prepare input matrices
x <- model.matrix(y ~ ., data = train)[, -1] # remove intercept column
y <- train$y

set.seed(123)

# Fit ridge with cross-validation
ridge_cv <- cv.glmnet(
  x,
  y,
  alpha = 0, # ridge penalty
  family = "binomial",
  type.measure = "auc" # use AUC for evaluation
)

# Best lambda
ridge_lambda <- ridge_cv$lambda.min
ridge_lambda

```

```
[1] 0.06214955
```

```

# Fit final ridge model at optimal lambda
ridge_fit <- glmnet(
  x,
  y,
  alpha = 0,
  lambda = ridge_lambda,
  family = "binomial"
)

# Predictions on test set
x_test <- model.matrix(y ~ ., data = test)[, -1]
ridge_prob <- predict(ridge_fit, newx = x_test, type = "response")

# Compute AUC
library(pROC)
ridge_auc <- auc(test$y, ridge_prob)
ridge_auc

```

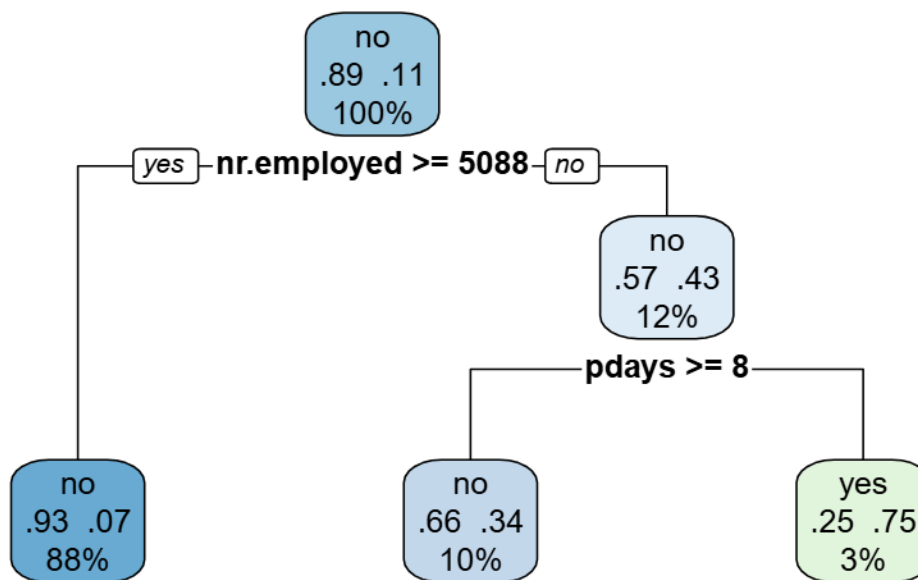
```
Area under the curve: 0.8163
```

Classification Tree

```
tree_fit <- rpart(y ~ ., data=train, method="class",
                  control=rpart.control(cp=0.001))

# Choose best cp
best_cp <- tree_fit$cptable[which.min(tree_fit$cptable[, "xerror"]), "CP"]
tree_pruned <- prune(tree_fit, cp=best_cp)

rpart.plot(tree_pruned, type=2, extra=104)
```



```
tree_prob <- predict(tree_pruned, test, type="prob")[, "yes"]
tree_pred <- ifelse(tree_prob > 0.5, "yes", "no") |> factor(levels=c("no", "yes"))

table(tree_pred, test$y)
```

```
tree_pred  no  yes
no      1103  101
yes       11   21
```



```
tree_auc <- auc(roc(test$y, tree_prob))
tree_auc
```

Area under the curve: 0.7547

No. of Terminal Nodes:

```
sum(tree_pruned$frame$var == "<leaf>")
```

```
[1] 3
```

Random Forest

```
set.seed(123)
rf_fit <- randomForest(y ~ ., data=train,
                      ntree=500,
                      mtry=floor(sqrt(ncol(train)-1)),
                      importance=TRUE)

rf_prob <- predict(rf_fit, test, type="prob")[, "yes"]
rf_pred <- predict(rf_fit, test)

table(rf_pred, test$y)
```

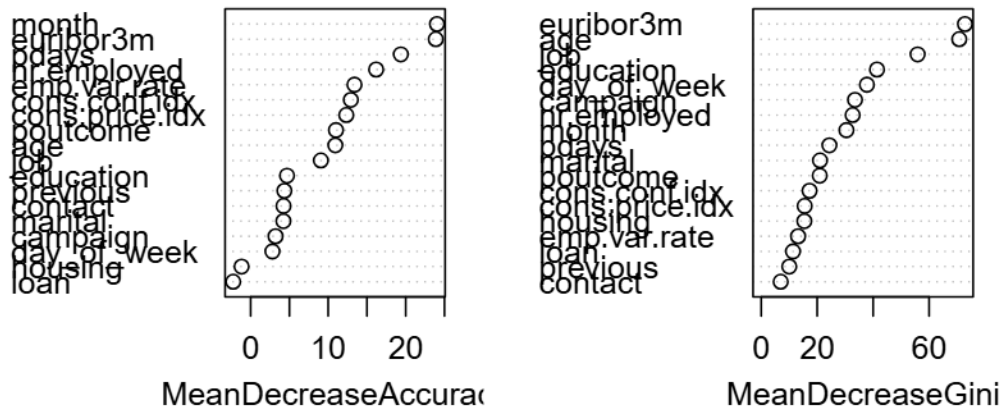
```
rf_pred  no  yes
      no 1080  85
      yes   34  37
```

```
rf_auc <- auc(roc(test$y, rf_prob))
rf_auc
```

Area under the curve: 0.8227

```
varImpPlot(rf_fit)
```

rf_fit



Model Performance Comparison

```
perf <- data.frame(
  Model = c("Logistic", "LASSO", "Tree", "Random Forest"),
  AUC = c(logit_auc, lasso_auc, tree_auc, rf_auc)
)
perf
```

	Model	AUC
1	Logistic	0.8071563
2	LASSO	0.8184875
3	Tree	0.7546686
4	Random Forest	0.8227146

Performance Interpretation

```
exp(coef(logit_fit))
```

(Intercept)	age
4.814691e-113	1.014749e+00
jobblue-collar	jobentrepreneur
8.006744e-01	3.970800e-01
jobhousemaid	jobmanagement
1.116469e+00	6.114672e-01
jobretired	jobself-employed
9.784843e-01	5.662718e-01
jobservices	jobstudent
1.138363e+00	9.443798e-01
jobtechnician	jobunemployed
8.870569e-01	1.091430e+00
jobunknown	maritalmarried
3.408572e-01	1.098294e+00
maritalsingle	maritalunknown
1.508378e+00	1.854267e+00
educationbasic.6y	educationbasic.9y
1.001434e+00	1.266847e+00
educationhigh.school	educationilliterate
9.387155e-01	1.367184e-05
educationprofessional.course	educationuniversity.degree
1.320530e+00	1.118270e+00
educationunknown	housingunknown
1.129274e+00	8.809349e-01
housingyes	loanunknown
9.169321e-01	NA
loanyes	contacttelephone
1.041100e+00	3.757855e-01
monthaug	monthdec
8.773028e-01	2.088888e+00
monthjul	monthjun
7.286778e-01	6.677372e-01
monthmar	monthmay
6.687229e+00	6.807859e-01
monthnov	monthoct
4.382942e-01	1.414149e+00
monthsep	day_of_weekmon
1.252425e+00	1.034217e+00
day_of_weekthu	day_of_weektue

1.118707e+00	9.316115e-01
day_of_weekwed	campaign
1.337444e+00	9.273526e-01
pdays	previous
1.000282e+00	1.139395e+00
poutcomenonexistent	poutcomesuccess
1.786078e+00	8.816545e+00
emp.var.rate	cons.price.idx
2.968441e-01	8.683581e+00
cons.conf.idx	euribor3m
1.063459e+00	8.929784e-01
nr.employed	
1.010926e+00	

```
coef(lasso_fit)
```

52 x 1 sparse Matrix of class "dgCMatrix"

	s0
(Intercept)	42.155070768
age	0.004620045
jobadmin.	.
jobblue-collar	.
jobentrepreneur	-0.214687945
jobhousemaid	.
jobmanagement	-0.071936210
jobretired	0.096923303
jobself-employed	-0.025892507
jobservices	.
jobstudent	.
jobtechnician	.
jobunemployed	.
jobunknown	.
maritalmarried	.
maritalsingle	0.142361534
maritalunknown	.
educationbasic.6y	.
educationbasic.9y	.
educationhigh.school	.
educationilliterate	.
educationprofessional.course	.
educationuniversity.degree	.
educationunknown	.

housingunknown	.
housingyes	.
loanunknown	.
loanyes	.
contacttelephone	-0.180800394
monthaug	.
monthdec	0.370476663
monthjul	.
monthjun	0.385935917
monthmar	1.205111441
monthmay	-0.468629654
monthnov	-0.351986430
monthoct	0.300708180
monthsep	.
day_of_weekmon	.
day_of_weekthu	.
day_of_weektue	.
day_of_weekwed	0.032310665
campaign	-0.029892977
pdays	.
previous	.
poutcomenonexistent	0.042285733
poutcomesuccess	1.601887555
emp.var.rate	.
cons.price.idx	.
cons.conf.idx	0.004939232
euribor3m	.
nr.employed	-0.008584025

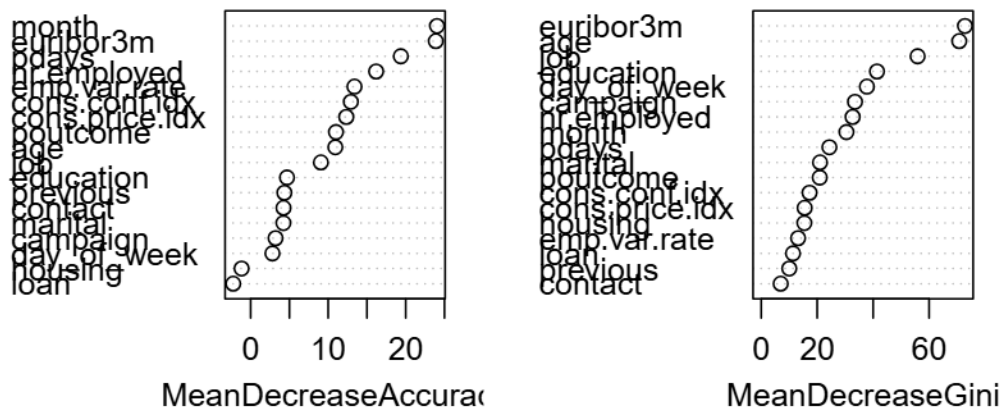
```
importance(rf_fit)
```

		no	yes	MeanDecreaseAccuracy	MeanDecreaseGini
age	13.1509274	-3.7576442		10.919390	70.770475
job	10.2391788	-1.0830926		9.059914	55.897790
marital	6.1840899	-5.5574161		4.228243	21.012742
education	6.0010414	-3.3353367		4.686966	41.377067
housing	-0.4171736	-2.1751467		-1.169248	15.417423
loan	-2.0972561	-0.9015429		-2.248659	11.357479
contact	2.9834724	5.8047055		4.239663	6.960341
month	22.9849010	-0.5940914		24.033348	30.447505
day_of_week	4.8646750	-4.5412587		2.809687	37.780179
campaign	3.0235876	1.4372200		3.210856	33.513321

pdays	7.4939746	21.5287386	19.371746	24.362263
previous	3.5699424	1.7093060	4.347709	10.027479
poutcome	6.9595002	9.7811114	11.004831	20.913009
emp.var.rate	13.2712100	-3.0921539	13.391259	13.182845
cons.price.idx	12.7098656	-9.5094783	12.336189	15.527606
cons.conf.idx	12.8259009	-5.5363647	12.922811	17.275517
euribor3m	22.6449087	-7.2364527	23.881558	72.822658
nr.employed	14.1271285	9.9397297	16.184809	32.676205

```
varImpPlot(rf_fit)
```

rf_fit



Bootstrap for stability (AUC of logistic regression)

```
logit_boot_auc <- function(data, indices) {
  # Bootstrap sample
  d <- data[indices, ]

  # Fit logistic regression on bootstrap sample
  fit <- glm(y ~ ., data = d, family = binomial)
```

```

# Predict on the same fixed test set to keep comparison fair
probs <- predict(fit, newdata = test, type = "response")

# Return AUC as a single number
auc(roc(test$y, probs))
}

set.seed(123)
boot_res <- boot(data = train, statistic = logit_boot_auc, R = 500)

boot_res

```

ORDINARY NONPARAMETRIC BOOTSTRAP

Call:

```
boot(data = train, statistic = logit_boot_auc, R = 500)
```

Bootstrap Statistics :

	original	bias	std. error
t1*	0.8071563	-0.01618919	0.01436831

```

# Bootstrap CI for AUC
boot.ci(boot_res, type = "perc")

```

BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS

Based on 500 bootstrap replicates

CALL :

```
boot.ci(boot.out = boot_res, type = "perc")
```

Intervals :

Level	Percentile
-------	------------

95%	(0.7600, 0.8169)
-----	--------------------

Calculations and Intervals on Original Scale